

TAREA 4

Diseño de Microservicios

API REST de Productos

Tipo de entrega	Documento de diseño
Alcance	Arquitectura, comunicación y justificación
Implementación	No se implementan microservicios en esta tarea

Documento preparado a partir de la arquitectura actual del proyecto, manteniendo como referencia la separación existente entre autenticación y catálogo de productos.

1. Contexto y objetivo

El proyecto actual consiste en una API REST de productos con autenticación mediante JWT, validación de datos y persistencia en MongoDB. En esta etapa no se agregará nueva funcionalidad: el objetivo es analizar cómo podría dividirse la aplicación en servicios independientes a medida que crezca.

La propuesta establece límites de responsabilidad claros, de modo que cada servicio se ocupe de un dominio concreto y pueda evolucionar, desplegarse y escalarse de forma independiente.

2. Servicios propuestos

2.1 Servicio de Autenticación

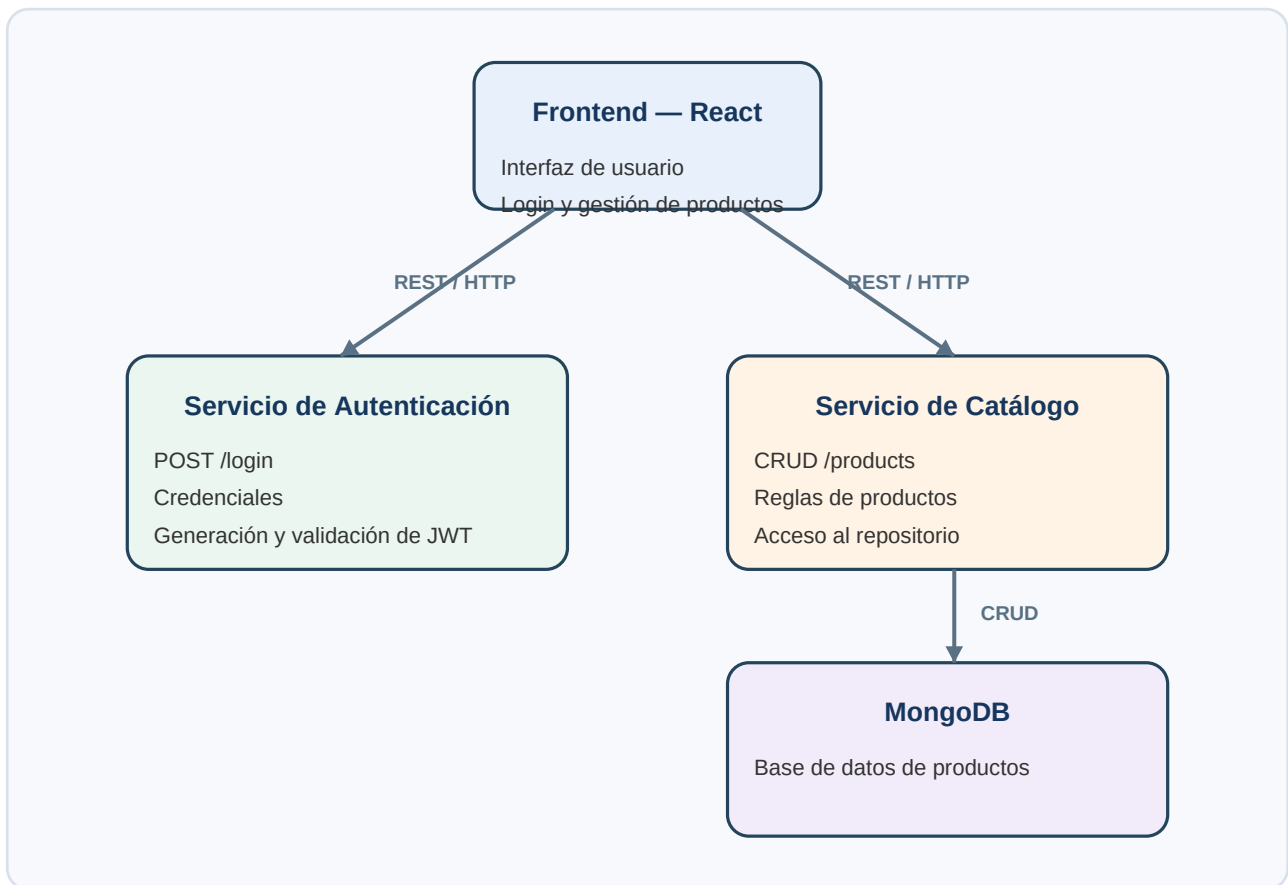
- Gestiona el inicio de sesión mediante **POST /login**.
- Valida las credenciales del usuario.
- Genera tokens JWT para que los clientes puedan autenticarse.
- Valida la autenticidad y vigencia de los tokens utilizados por otros servicios.
- Su responsabilidad se limita a identidad y autorización.

2.2 Servicio de Catálogo de Productos

- Gestiona el dominio de productos.
- Expone las operaciones CRUD: crear, consultar, actualizar y eliminar productos.
- Aplica las reglas de validación propias de los productos.
- Utiliza un repositorio para comunicarse con MongoDB.
- No contiene la lógica de autenticación como responsabilidad principal.

3. Diagrama de arquitectura

El siguiente diagrama muestra una separación conceptual del sistema. El frontend consume los servicios mediante HTTP/REST; el servicio de catálogo se encarga de la persistencia de los productos.



4. Tipo de comunicación

Se propone comunicación síncrona mediante REST/HTTP. Para el alcance actual, esta opción es la más sencilla y adecuada porque las operaciones principales requieren una respuesta inmediata. Por ejemplo, cuando un usuario inicia sesión, el frontend necesita recibir el JWT antes de continuar; de igual forma, cuando consulta o crea un producto, necesita conocer inmediatamente el resultado de la operación.

La mensajería asíncrona podría incorporarse posteriormente para procesos que no necesiten respuesta inmediata, como notificaciones, auditoría, generación de reportes o integración con otros sistemas. Sin embargo, introducir un broker de mensajes desde esta etapa agregaría complejidad sin ser necesario para el flujo principal.

5. ¿Qué problema habría si no se divide?

Si el proyecto continúa creciendo como un único servicio, autenticación, productos y nuevas funcionalidades pueden quedar cada vez más acoplados. Esto puede hacer que el código sea más difícil de mantener, que los cambios en un módulo afecten a otros y que los despliegues deban realizarse sobre toda la aplicación. También sería más difícil escalar únicamente la parte que recibe mayor carga. La división por servicios permite establecer responsabilidades claras y facilita que cada dominio pueda evolucionar y escalar de manera independiente.

6. Justificación de los límites

Servicio	Responsabilidad principal	Beneficio del límite
Autenticación	Identidad, login y JWT	Aísla seguridad y autorización del dominio de productos.
Catálogo	Productos y persistencia	Permite evolucionar y escalar el catálogo independientemente.

Nota: dividir un sistema en microservicios también introduce costos adicionales, como despliegues independientes, observabilidad, configuración de red y manejo de errores distribuidos. Por eso, la división propuesta se limita a dos dominios claros y evita crear servicios demasiado pequeños sin una necesidad real.

7. Estructura para el repositorio

El documento debe almacenarse dentro de una carpeta **/docs** en la raíz del repositorio. Se recomienda la siguiente estructura:

```

Api-Rest-de-Productos/
  ■■■ backend/
  ■■■ frontend/
  ■■■ docs/
  ■ ■■■ Tarea_4_Diseño_de_Microservicios.pdf
  ■■■ README.md
  ■■■ .gitignore

```

Una vez colocado el archivo, se puede registrar en Git con un commit específico para la Tarea 4.

8. Conclusión

La propuesta divide el sistema en dos servicios con límites claros: autenticación y catálogo de productos. La comunicación síncrona mediante REST mantiene el flujo sencillo y responde a las necesidades actuales del proyecto. Esta separación constituye una base para una futura implementación de microservicios, sin realizar todavía cambios de código, tal como establece la consigna de la Tarea 4.